

Advanced Design Scripting & Parametrics – Class 07 – Meshes and BREPs

Today – key concepts

- Meshes
- Boundary Representations (BREPS)
- Surface curvature if we have time

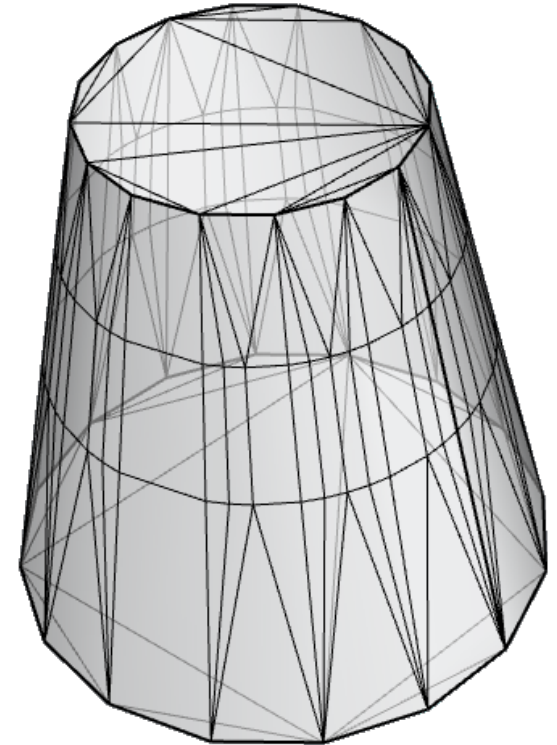
Meshes

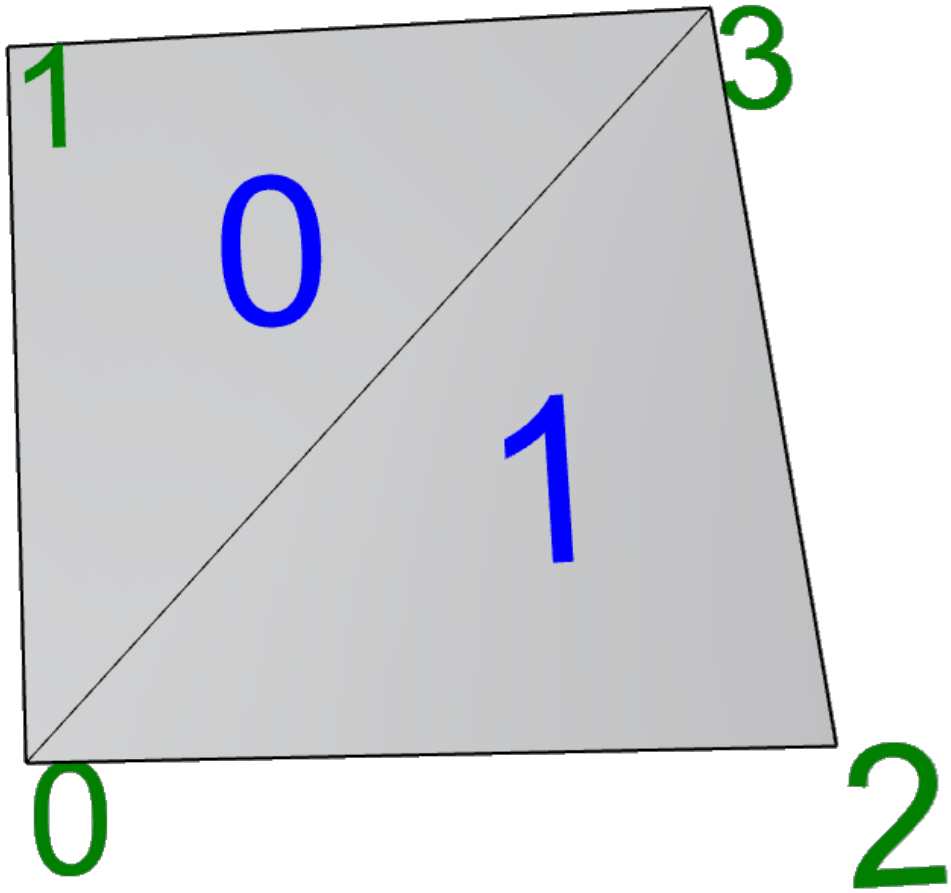
Meshes are collections of triangles (and sometimes “quads”), assembled into contiguous closed or open configurations.

Several triangles can share a common vertex, so there are efficient ways of making these references between triangles and vertices.

Vertices can have additional information, including:

- Color
- Effective normal
- Positioning in 2D texture space





Mesh elements

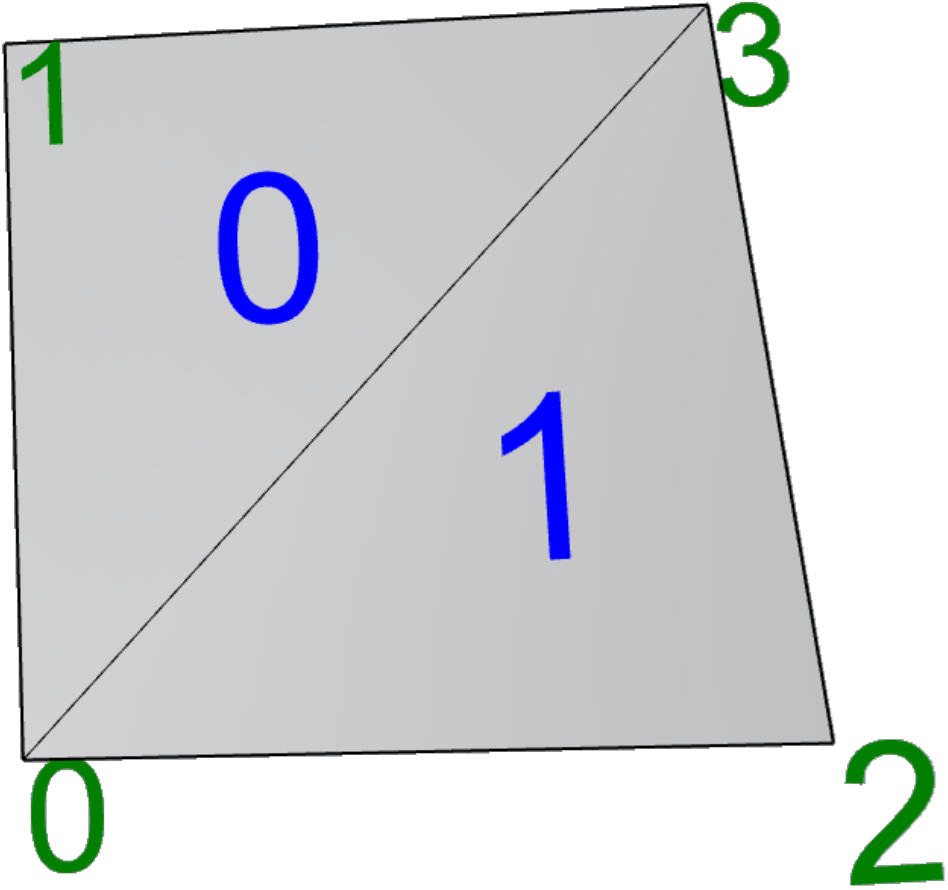
Comprised of:

- **Vertices** – each has:
 - an **index** (integer) corresponding to its index in the vertex arrayand optionally:
 - A **normal** (Vector3f)
 - A **texture coordinate** (Point2f)
 - A **color** (Color)
- **Faces** - each has
 - 4 vertex **indices** A, B, C, D (integers), where the C and D can be duplicates can be duplicates
 - An implied surface **normal** (Vector3f), with a direction “pointing out” defined by the cross product of the vectors AB and BC
- **Edges** – implied, which can be:
 - Internal
 - “Naked”
 - Non-manifold

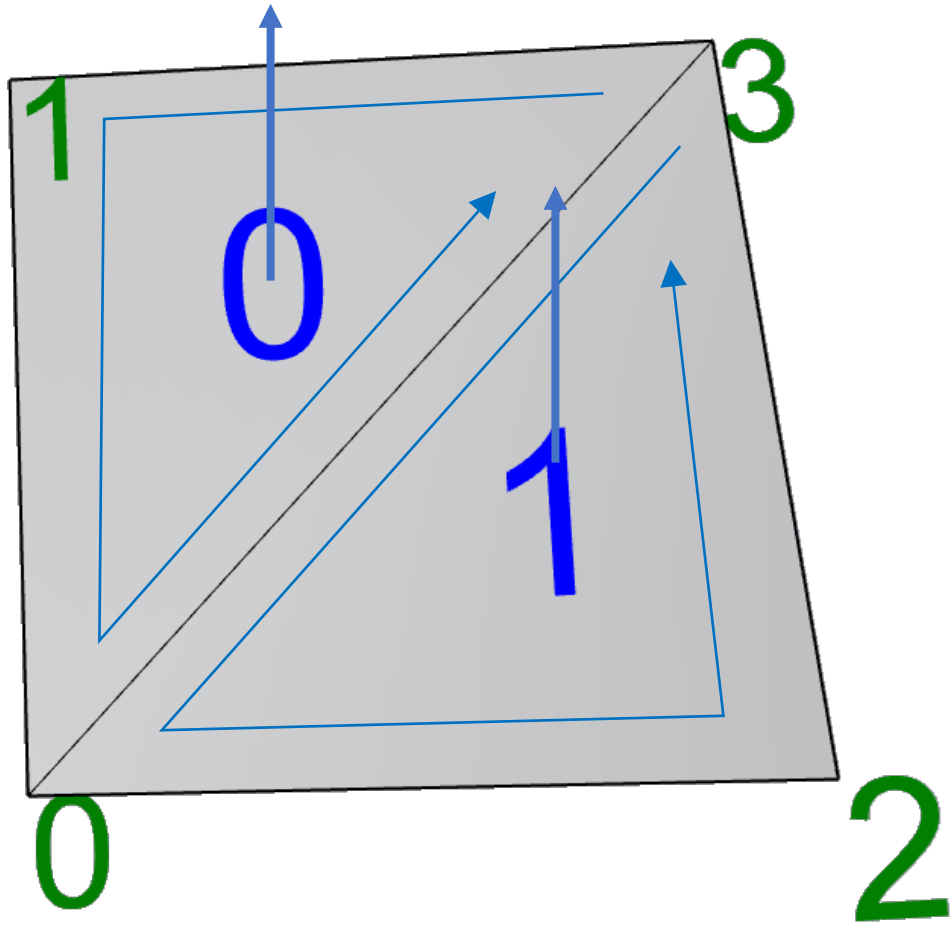
Mesh elements

A mesh can be:

- Open
- Closed
- Non-manifold (almost never is)



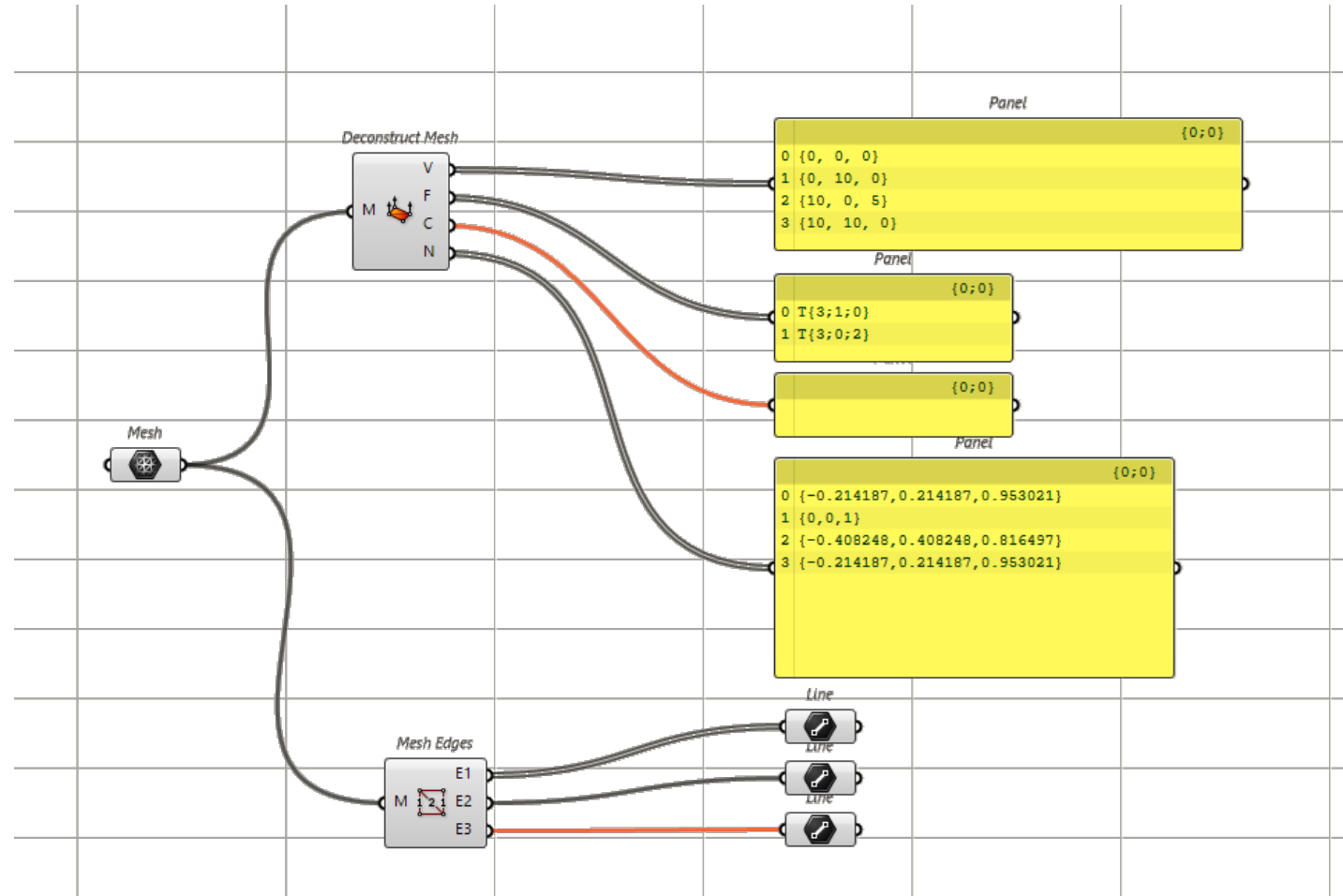
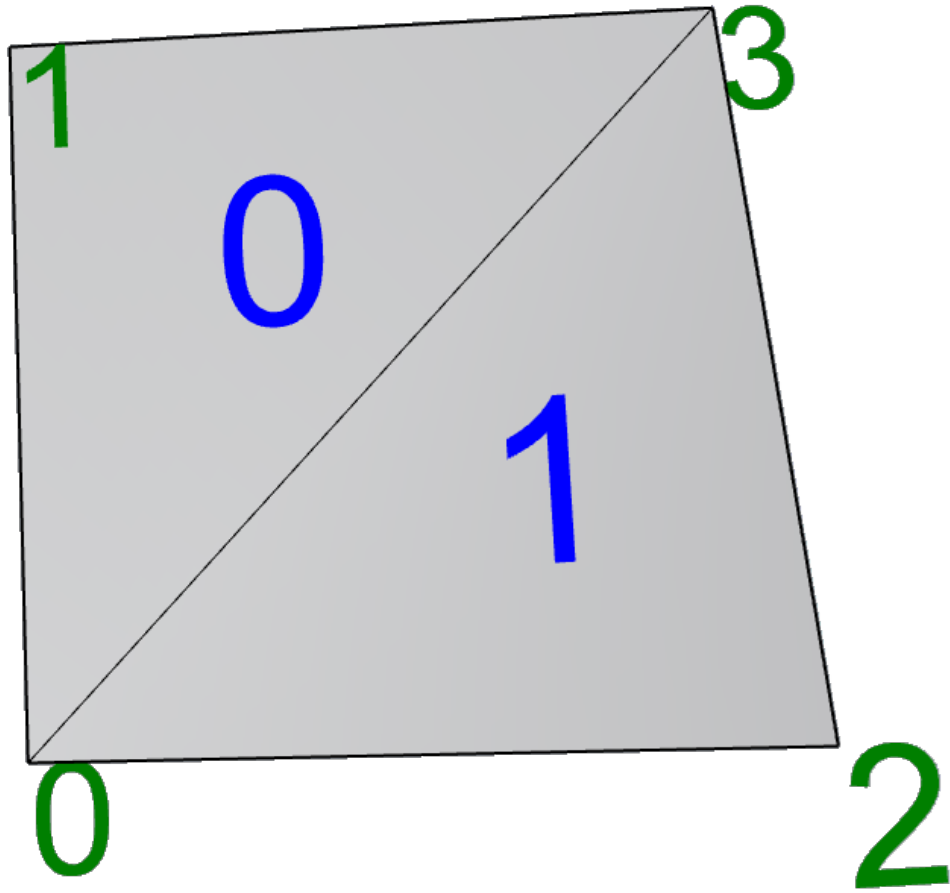
AnalyzeMesh.py



▲ vertices	4 Item(s)	list
▸ [0]	0,0,0	Point3f
▸ [1]	0,10,0	Point3f
▸ [2]	10,0,5	Point3f
▸ [3]	10,10,0	Point3f
▲ faces	2 Item(s)	MeshFace
▲ [0]	T(3; 1, 0)	MeshFace
A	3	int
B	1	int
C	0	int
D	0	int
IsQuad	False	bool
IsTriangle	True	bool
▸ Unset	T(-2147483648, -2147483648)	MeshFace
▲ [1]	T(3; 0, 2)	MeshFace
A	3	int
B	0	int
C	2	int
D	2	int
IsQuad	False	bool
IsTriangle	True	bool
▸ Unset	T(-2147483648, -2147483648)	MeshFace
Capacity	2	int
Count	2	int
QuadCount	0	int
TriangleCount	2	int

▲ normals	2 Item(s)	MeshFace
▸ [0]	0,0,1	Vector3f
▸ [1]	-0.4082483, 0.4082483, 0.8164966	Vector3f
Capacity	2	int
Count	2	int
▲ textureCoords	4 Item(s)	MeshText
▸ [0]	0,0	Point2f
▸ [1]	0,1	Point2f
▸ [2]	1,0	Point2f
▸ [3]	1,1	Point2f

AnalyzeMesh.gh



▼ Mesh

► Constructors

▼ Properties

- ComponentStates
- DisjointMeshCount
- Disposed
- FaceNormals
- Faces
- HasBrepForm
- HasCachedTextureCoordinates
- HasPrincipalCurvatures
- HasUserData
- IsClosed
- IsDeformable
- IsDocumentControlled
- IsDocumentControlled
- IsOriented
- IsSolid
- IsValid
- Ngons
- Normals
- ObjectType
- PartitionCount
- PerformCorruptionTesting
- TextureCoordinates
- TopologyEdges
- TopologyVertices
- UserData
- UserDictionary
- UserStringCount
- VertexColors
- Vertices

▼ Methods

► Append

- Check
- ClearSurfaceData
- ClearTextureData
- ClosestMeshPoint
- ClosestPoint
- CollapseFacesByArea
- CollapseFacesByByAspectRatio
- CollapseFacesByEdgeLength
- ColorAt
- Compact
- ComponentIndex
- ComputeAutoCreaseInformation
- ComputeCurvatureApproximation
- ComputeThickness
- ConstructConstObject
- CopyFrom
- CreateBooleanDifference
- CreateBooleanIntersection
- CreateBooleanSplit
- CreateBooleanUnion
- CreateContourCurves
- CreateConvexHull3D
- CreateExtrusion
- CreateFromBox
- CreateFromBrep
- CreateFromClosedPolyline
- CreateFromCone
- CreateFromCurveExtrusion
- CreateFromCurvePipe
- CreateFromCylinder
- CreateFromExtrusion
- CreateFromFilteredFaceList
- CreateFromIterativeCleanup
- CreateFromLines
- CreateFromPatchSingleFace
- CreateFromPlanarBoundary
- CreateFromPlane
- CreateFromSphere

- CreateFromSubD
- CreateFromSubDControlNet
- CreateFromSubDControlNetWithTextureCoordinates
- CreateFromSurface
- CreateFromSurfaceControlNet
- CreateFromTessellation
- CreateFromTorus
- CreateIcoSphere
- CreatePartitions
- CreatePatch
- CreateQuadSphere
- CreateRefinedCatmullClarkMesh
- CreateRefinedLoopMesh
- CreateUnweldedMesh
- CreateVertexColorsFromBitmap
- DataCRC
- DeleteAllUserStrings
- DeleteUserString
- DestroyPartition
- DestroyTopology
- DestroyTree
- Dispose
- Dispose
- Duplicate
- Duplicate
- DuplicateMesh
- DuplicateShallow
- EnsurePrivateCopy
- EvaluateMeshGeometry
- ExplodeAtUnweldedEdges
- ExtendSelectionByEdgeRidge
- ExtendSelectionByFaceLoop
- ExtractNonManifoldEdges
- FileHole
- FillHoles
- Flip
- FromBase64String
- FromJSON
- GeometryEquals
- GeometryReferenceEquals
- GetBoundingBox
- GetCachedTextureCoordinates
- GetNakedEdgePointStatus
- GetNakedEdges

- GetNgonAndFacesCount
- GetNgonAndFacesEnumerable
- GetObjectData
- GetOutlines
- GetPartition
- GetSelfIntersections
- GetUnsafeLock
- GetUserString
- GetUserStrings
- HealNakedEdges
- InvalidateCachedTextureCoordinates
- IsManifold
- IsPointInside
- IsValidWithLog
- MakeDeformable
- MatchEdges
- MemoryEstimate
- MergeAllCoplanarFaces
- NonConstOperation
- NonConstOperation
- NonConstOperation
- ▼ NormalAt
 - (int, double, double, double, double)
 - (MeshPoint)
- Offset
- OnSwitchToNonConst
- OnSwitchToNonConst
- OnSwitchToNonConst
- PatchSingleFace
- PointAt
- PullCurve
- PullPointsToMesh
- QuadRemesh
- QuadRemeshAsync
- QuadRemeshBrep
- QuadRemeshBrepAsync
- RebuildNormals
- Reduce
- ReleaseUnsafeLock
- RequireIterativeCleanup
- Rotate
- Scale
- SetCachedTextureCoordinates
- SetCachedTextureCoordinatesFromMaterial

- SetTextureCoordinates
- SetUserString
- ShrinkWrap
- Smooth
- SolidOrientation
- Split
- SplitDisjointPieces
- SplitWithProjectedPolylines
- Subdivide
- ToJSON
- Transform
- Translate
- UnifyNormals
- Unweld
- UnweldEdge
- UnweldVertices
- Volume
- Weld
- WithDisplacement
- WithEdgeSoftening
- WithShutLining

Mesh Geometries

- ▶ Mesh
- ▶ MeshBooleanOptions
- ▶ MeshCheckParameters
- ▶ MeshDisplacementInfo
- ▶ MeshExtruder
- MeshExtruderFaceDirectionMode
- MeshExtruderParameterMode
- ▶ MeshFace
- ▶ MeshingParameters
- MeshingParameterStyle
- MeshingParameterTextureRange
- ▶ MeshNgon
- ▶ MeshPart
- MeshPipeCapStyle
- ▶ MeshPoint
- ▶ MeshThicknessMeasurement
- MeshType
- ▶ MeshUnsafeLock
- MeshUnwrapMethod
- ▶ MeshUnwrapper

▼ MeshFace

▶ Constructors

▼ Properties

A

B

C

D

IsQuad

IsTriangle

Item

Unset

▼ Methods

CompareTo

▶ Equals

Flip

GetHashCode

▶ IsValid

IsValidEx

Repair

RepairEx

▶ Set

ToString

▼ Operators

!=

==

Mesh Collections

- ▶ MeshFaceList
- ▶ MeshFaceNormalList
- ▶ MeshNgonList
- ▶ MeshTextureCoordinateList
- ▶ MeshTopologyEdgeList
- ▶ MeshTopologyVertexList
- ▶ MeshVertexColorList
- ▶ MeshVertexList
- ▶ MeshVertexNormalList
- ▶ MeshVertexStatusList

▼ MeshVertexList

- ▼ Properties
 - Capacity
 - Count
 - Item **Point3f**
 - UseDoublePrecisionVertices
- ▼ Methods
 - ▶ Add
 - ▶ AddVertices
 - ▶ Align
 - Clear
 - CombineIdentical
 - CullUnused
 - Destroy
 - GetConnectedVertices
 - GetEnumerator
 - GetTopologicallyIdenticalVertices
 - GetVertexFaces
 - Hide
 - HideAll
 - IsHidden
 - Point3dAt
 - ▶ Remove
 - ▶ SetVertex
 - Show
 - ShowAll
 - ToFloatArray
 - ToPoint3dArray
 - ToPoint3fArray

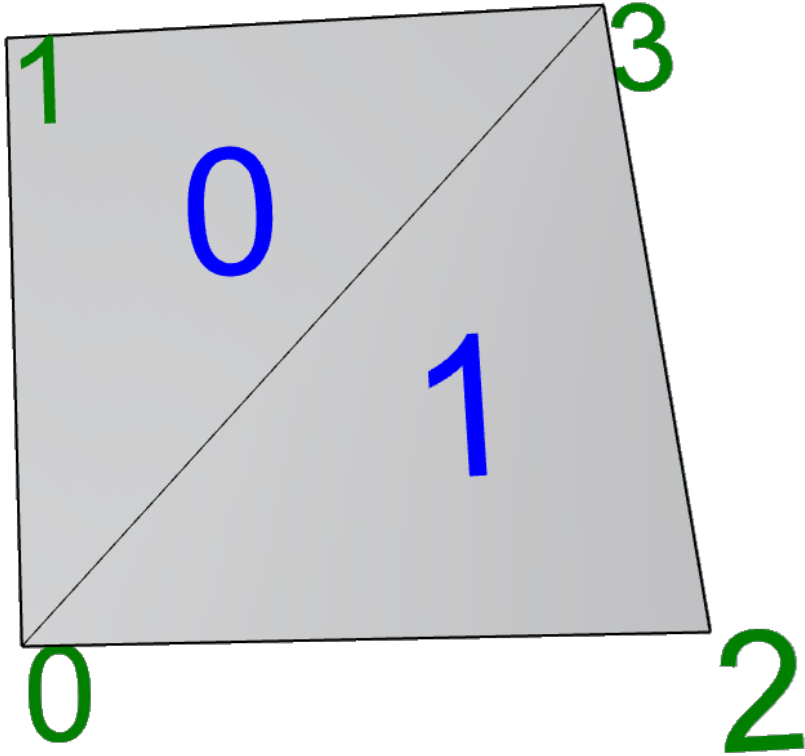
▼ MeshVertexColorList

- ▼ Properties
 - Capacity
 - Count
 - Item **Color**
 - Tag
- ▼ Methods
 - ▶ Add
 - AddRange
 - AppendColors
 - Clear
 - CreateMonotoneMesh
 - Destroy
 - GetEnumerator
 - ▶ SetColor
 - SetColors
 - ToARGBArray

▼ MeshVertexNormalList

- ▼ Properties
 - Capacity
 - Count
 - Item **Vector3f**
- ▼ Methods
 - ▶ Add
 - AddRange
 - Clear
 - ComputeNormals
 - Destroy
 - Flip
 - GetEnumerator
 - ▶ SetNormal
 - SetNormals
 - ToFloatArray
 - UnitizeNormals

OBJ Definition ("1 Ordered")



.OBJ file

```
1 # Rhino
2 |
3 mtllib smallmesh.mtl
4 usemtl diffuse_0_0_0_255
5 v 0 0 0
6 v 0 10 0
7 v 10 0 5
8 v 10 10 0
9 vt 0 0
10 vt 0 1
11 vt 1 0
12 vt 1 1
13 vn -0.214 0.214 0.953
14 vn 0 0 1
15 vn -0.408 0.408 0.816
16 vn -0.214 0.214 0.953
17 f 4/4/4 2/2/2 1/1/1
18 f 4/4/4 1/1/1 3/3/3
```

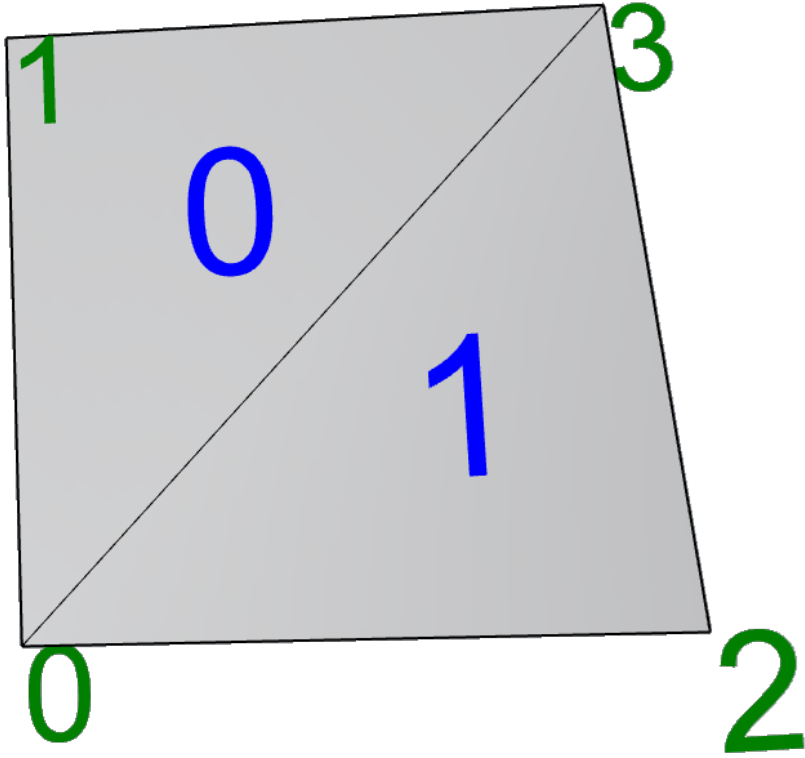
vertices {
texture {
faces normals {

↑ ↑ ↑
v t n

.MTL file

```
1 # Rhino
2 newmtl diffuse_0_0_0_255
3 Ka 0.0000 0.0000 0.0000
4 Kd 0.0000 0.0000 0.0000
5 Ks 1.0000 1.0000 1.0000
6 Tf 1.0000 1.0000 1.0000
7 d 1.0000
8 Ns 0.0000
9 Ni 1.0000
```

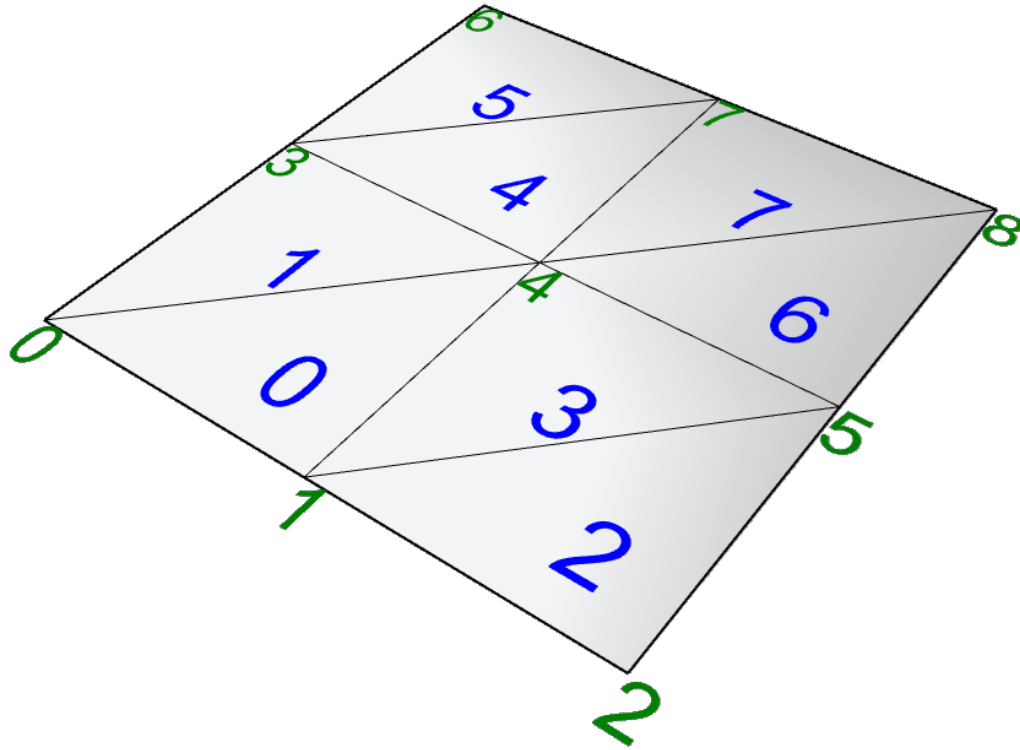
STL Definition



.STL file

```
1  ~ solid OBJECT
2  ~   facet normal 0 -0 1
3  ~     outer loop
4      vertex 10 10 0
5      vertex 0 10 0
6      vertex 0 0 0
7     endloop
8   endfacet
9  ~   facet normal -0.408 0.408 0.816
10 ~     outer loop
11      vertex 10 10 0
12      vertex 0 0 0
13      vertex 10 0 5
14     endloop
15   endfacet
16 endsolid OBJECT
```

Making a mesh (makeMesh01.gh)



index

ranges

vertices

faces

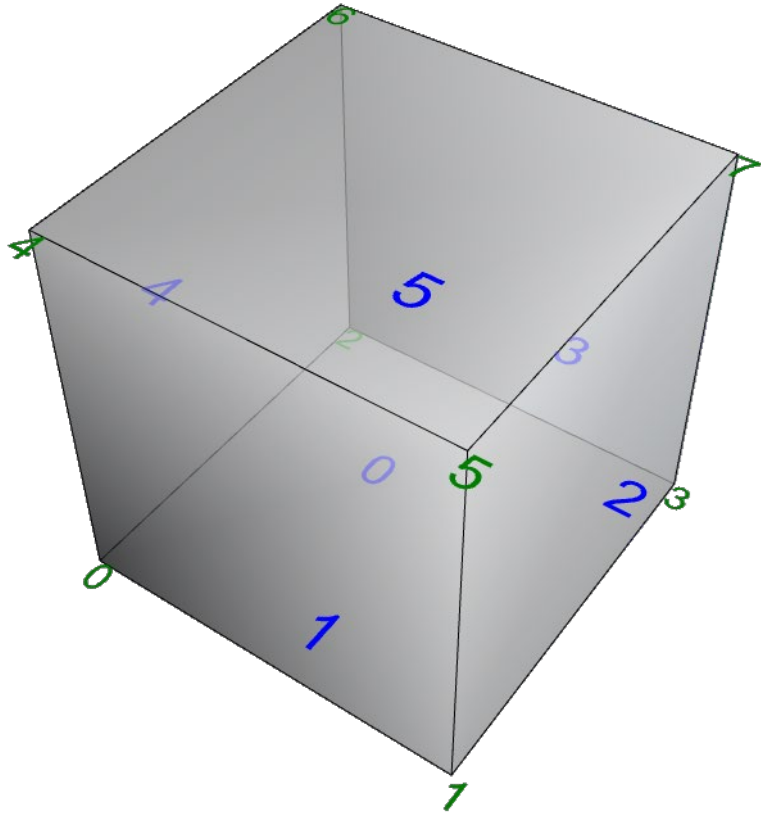
mesh

```

11 #index helper functions
12 def VID(i, j):
13     return j * countX + i
14 def V(i, j):
15     return vertices(VID(i, j))
16
17 #make x values
18 xvals = numpy.arange(0, sizeX, sizeX / float(countX - 1))
19 xvals = numpy.append(xvals, sizeX)
20 #print(xvals)
21 countX = xvals.size
22
23 #make y values
24 yvals = numpy.arange(0, sizeY, sizeY / float(countY - 1))
25 yvals = numpy.append(yvals, sizeY)
26 yCount = yvals.size
27
28 #create the vertices that the edges will reference later
29 vertices = []
30 for y in yvals:
31     i = 0
32     for x in xvals:
33         vertices.append((x,y,0.0))
34
35 # make the face vertex lists
36 faceVertices = []
37 j = 0
38 for y in yvals:
39     i = 0
40     for x in xvals:
41         if (i < countX - 1) and (j < countY - 1):
42             ##### forward faces
43             faceVertices.append((VID(i,j), VID(i+1,j), VID(i+1,j+1)))
44             faceVertices.append((VID(i,j), VID(i+1,j+1), VID(i,j+1)))
45             i = i + 1
46         j = j + 1
47
48 #Make the mesh
49 mesh = rs.AddMesh( vertices, faceVertices )

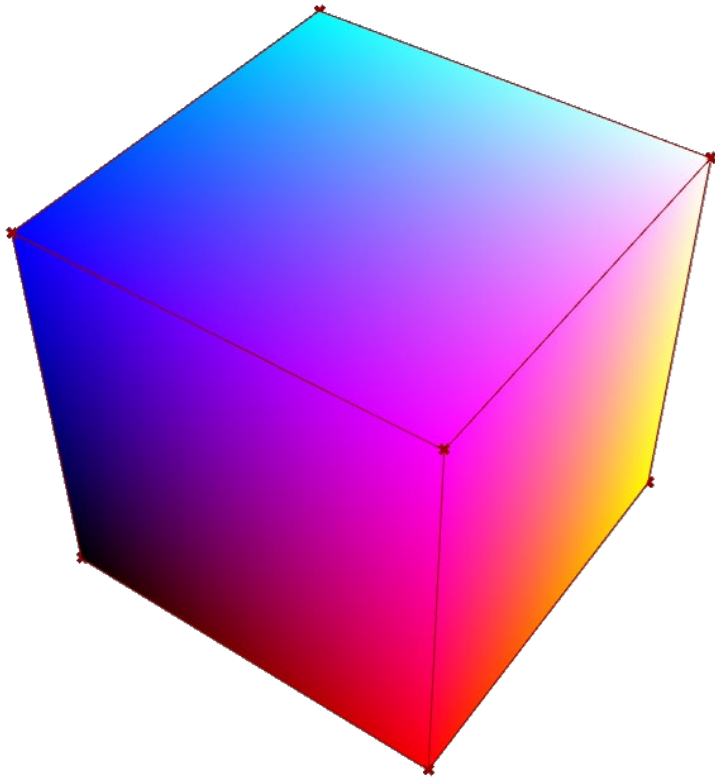
```

Making a cube (makeMesh04 – cube)



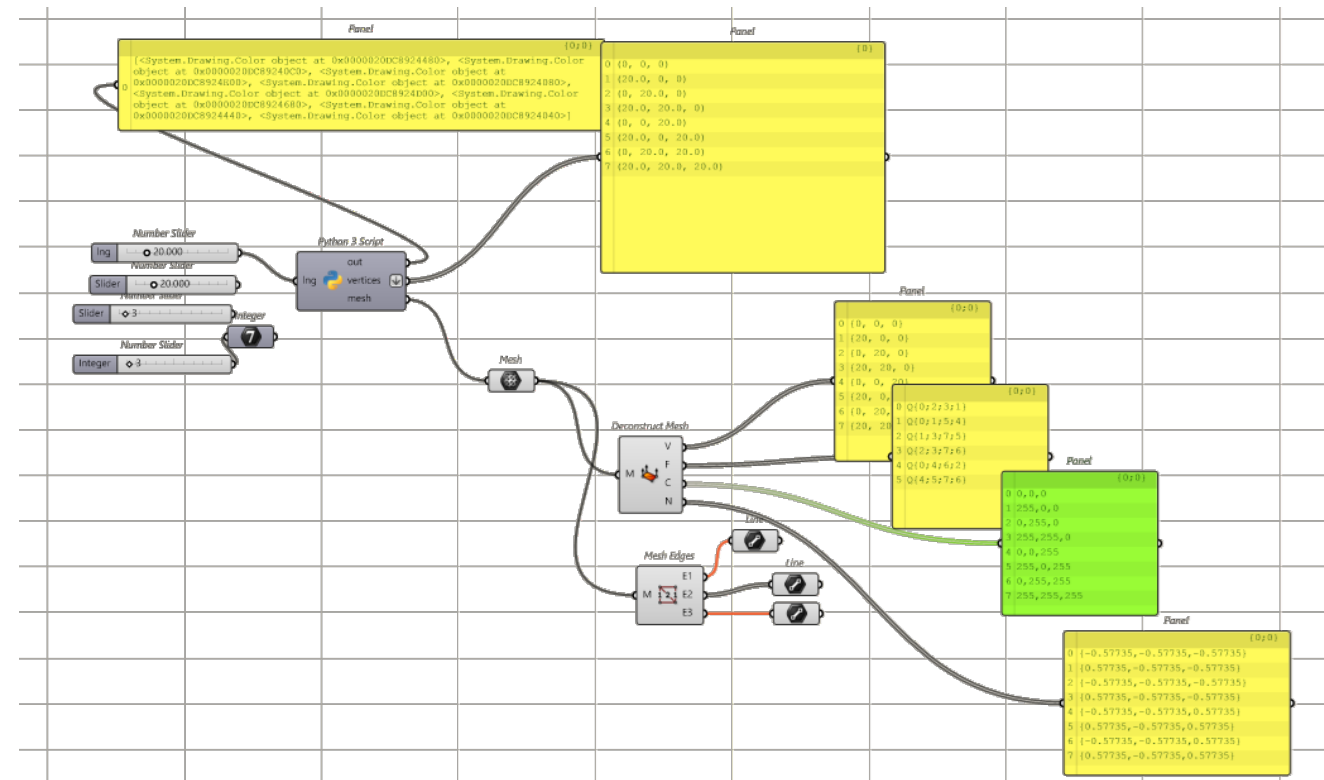
```
12 #create the vertices that the edges will reference later
13 vertices = []
14 vertices.append((0,0,0))
15 vertices.append((lng,0,0))
16 vertices.append((0,lng,0))
17 vertices.append((lng,lng,0))
18
19 vertices.append((0,0,lng))
20 vertices.append((lng,0,lng))
21 vertices.append((0,lng,lng))
22 vertices.append((lng,lng,lng))
23
24
25 faceVertices = []
26
27 faceVertices.append((0,2,3,1)) #bottom
28 faceVertices.append((0,1,5,4)) #front
29 faceVertices.append((1,3,7,5)) #right
30 faceVertices.append((2,3,7,6)) #back
31 faceVertices.append((0,4,6,2)) #left
32 faceVertices.append((4,5,7,6)) #top
33
34
35 #Make the mesh
36 mesh = rs.AddMesh( vertices, faceVertices )
```

Making a cube with colors (makeMesh05 – cube)



```

36 vcolors = []
37 for i in range(8):
38     vcolors.append(rs.CreateColor(int(vertices[i][0] / lng * 255), \
39     int(vertices[i][1] / lng * 255), \
40     int(vertices[i][2] / lng * 255)))
41
42 print(vcolors)
43 #Make the mesh
44 mesh = rs.AddMesh( vertices, faceVertices, vertex_colors = vcolors )
    
```



BREPs (Boundary Representations)

BREPs are collections of trimmed or untrimmed NURBS surfaces, that are connected into open or closed objects.

In Rhino, Solids are closed BREPS

